

**Université M'Hamed Bougara**  
Faculté de Technologie de Boumerdès

Génie des Systèmes Electriques

---

# SplashVision

Votre main est la seule manette

Projet Python – Version 1.0

---

## Informations du Projet

|                     |                                |
|---------------------|--------------------------------|
| Réalisé par         | Saada Abderrahmane             |
| Et                  | Halimi Nour El Islam           |
| Encadré par         | Mme. Saibi                     |
| Département         | Génie des Systèmes Electriques |
| Section             | AUTOMATIQUE MCIL               |
| Groupe              | 2                              |
| Année universitaire | 2025–2026                      |

## Table des matières

---

|                                               |          |
|-----------------------------------------------|----------|
| <b>Résumé</b>                                 | <b>2</b> |
| <b>1 Introduction</b>                         | <b>3</b> |
| <b>2 Genèse et Objectifs</b>                  | <b>3</b> |
| 2.1 Pourquoi SplashVision? . . . . .          | 3        |
| 2.2 Public cible . . . . .                    | 3        |
| <b>3 Vue d'ensemble du Gameplay</b>           | <b>3</b> |
| <b>4 Modes de Jeu</b>                         | <b>4</b> |
| <b>5 Interface Homme-Machine sans Contact</b> | <b>4</b> |
| 5.1 Curseur et Squelette . . . . .            | 4        |
| 5.2 Boutons Hover-to-Select . . . . .         | 4        |
| 5.3 Écrans . . . . .                          | 5        |
| <b>6 Technologies et Algorithmes</b>          | <b>5</b> |
| 6.1 Vision par Ordinateur . . . . .           | 5        |
| 6.2 Détection de Coupe . . . . .              | 5        |
| 6.3 Moteur de Jeu . . . . .                   | 5        |
| 6.4 Physique . . . . .                        | 6        |
| <b>7 Architecture Logicielle</b>              | <b>6</b> |
| <b>8 Performances et Optimisations</b>        | <b>6</b> |
| <b>9 Retours Sensoriels</b>                   | <b>6</b> |
| 9.1 Visuels . . . . .                         | 6        |
| 9.2 Audio . . . . .                           | 7        |
| <b>10 Guide de Prise en Main</b>              | <b>7</b> |
| <b>11 Personnalisation et Extensions</b>      | <b>7</b> |
| <b>12 Démonstration Visuelle</b>              | <b>8</b> |
| <b>13 Conclusion et Perspectives</b>          | <b>8</b> |
| <b>14 Remerciements et Crédits</b>            | <b>8</b> |

## Résumé

Ce document présente une étude approfondie du projet **SplashVision**, un jeu d'adresse 2D où l'interaction repose exclusivement sur la vision par ordinateur. Grâce à une simple webcam, le joueur contrôle un curseur avec le bout de son index et tranche des fruits virtuels par des gestes naturels. Ce projet illustre comment la détection de la main (MediaPipe) et le flux optique peuvent remplacer les périphériques traditionnels pour créer une expérience immersive.

L'analyse couvre l'architecture logicielle modulaire, les performances optimisées (60 FPS), la double détection de coupe, les modes de jeu et l'interface sans contact. Ce système constitue une démonstration concrète des capacités de la vision par ordinateur open source pour l'interaction homme-machine.

**Mots-clés :** Vision par Ordinateur, Détection de main, MediaPipe, OpenCV, Pygame, Flux optique, Temps Réel

## 1 Introduction

---

**SplashVision** est un jeu d'adresse 2D où l'interaction repose exclusivement sur la **vision par ordinateur**. Grâce à une simple webcam, le joueur contrôle un curseur avec le bout de son index et tranche des fruits virtuels par des gestes naturels. Ce projet illustre comment la **détection de la main** et le **flux optique** peuvent se substituer aux périphériques traditionnels (souris, clavier, écran tactile) pour créer une expérience immersive, inclusive et ludique.

Cette présentation détaille l'ensemble des aspects du projet : concept, fonctionnalités, technologies, architecture, performances et perspectives.

## 2 Genèse et Objectifs

---

### 2.1 Pourquoi SplashVision ?

- **Innovation** : repousser les limites de l'interaction homme-machine en utilisant uniquement le corps.
- **Accessibilité** : jouable avec n'importe quelle webcam standard, sans matériel coûteux.
- **Éducation** : démontrer de manière concrète les capacités de MediaPipe et OpenCV.
- **Fun** : offrir un gameplay addictif inspiré de *Fruit Ninja*, mais en version « air-touch ».

### 2.2 Public cible

- Développeurs intéressés par la vision par ordinateur et le game design.
- Établissements scolaires pour des démonstrations interactives.
- Joueurs occasionnels cherchant une expérience originale.

## 3 Vue d'ensemble du Gameplay

---

Le joueur se tient face à sa webcam. L'image est affichée en **plein écran** avec, superposés, des **fruits** et des **bombes** en mouvement. La **main** du joueur est détectée et un squelette vert est dessiné sur l'écran. Le bout de l'**index** sert de curseur.

Pour **trancher un fruit**, il suffit de passer la main rapidement à l'endroit où il se trouve. La détection combine :

- La **vitesse du curseur** (index) pour des gestes vifs,
- Le **flux optique** (mouvement global des pixels) en secours.

Les **bombes** pénalisent le joueur : secousse d'écran, flash rouge, perte de points ou de temps.

Les **menus** sont entièrement navigables sans clic : le joueur maintient son index sur un bouton jusqu'à ce qu'une jauge circulaire se remplisse.

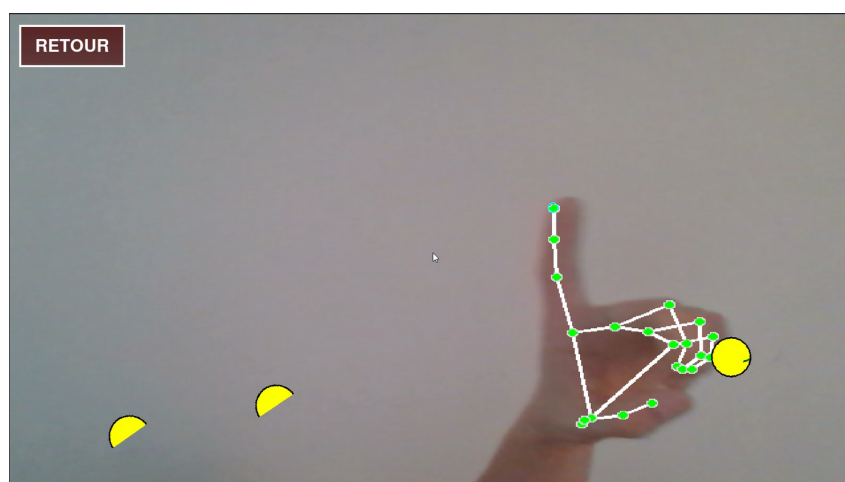


FIGURE 1 – Aperçu du gameplay

## 4 Modes de Jeu

TABLE 1 – Récapitulatif des modes de jeu

| Mode   | Durée        | Contenu          | Règles principales                                                                                                                         |
|--------|--------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| ZEN    | Illimitée    | Fruits seulement | Aucun score, aucune pression. Idéal pour s'entraîner.                                                                                      |
| ARCADE | Illimitée    | Fruits + Bombes  | Fruit = <b>+100 pts</b> , Bombe = <b>-200 pts</b> . Pas de limite de temps.                                                                |
| CHRONO | 120 secondes | Fruits + Bombes  | Fruit = <b>+100 pts</b> , Bombe = <b>-10 s</b> . Combo : 5 fruits sans erreur = <b>+5 s</b> . Le combo continue (bonus tous les 5 fruits). |

### Particularités :

- Aucune perte de vie quand un fruit tombe sans être coupé.
- En Chrono, un bip sonore retentit chaque seconde dans les 10 dernières secondes.
- Le combo n'est réinitialisé que par une bombe.

## 5 Interface Homme-Machine sans Contact

### 5.1 Curseur et Squelette

- **MediaPipe Hands** détecte la main et ses 21 points de repère.
- Le curseur est positionné sur le **landmark 8** (bout de l'index).
- Si la main n'est pas détectée, un fallback utilise le centroïde du **mouvement** (flux optique).

### 5.2 Boutons Hover-to-Select

- Des rectangles semi-transparents avec dégradé et contour épais.
- Jauge circulaire qui se remplit lors du maintien du curseur.

- Temps de maintien réglable (2 s pour les modes, 1,5 s pour Retour).
- Feedback sonore : son au survol (hover.wav), son à l'activation (click.wav).
- Boutons de couleur différente selon leur fonction (rouge foncé pour Retour/Quitter, bleu pour How-to).

### 5.3 Écrans

- **Menu principal** : 5 boutons (ZEN, ARCADE, CHRONO, HOW TO PLAY, QUITTER).
- **Écran How-to** : liste d'instructions, bouton BACK pour revenir.
- **Écran de jeu** : bouton RETOUR en haut à gauche pour interrompre la partie.

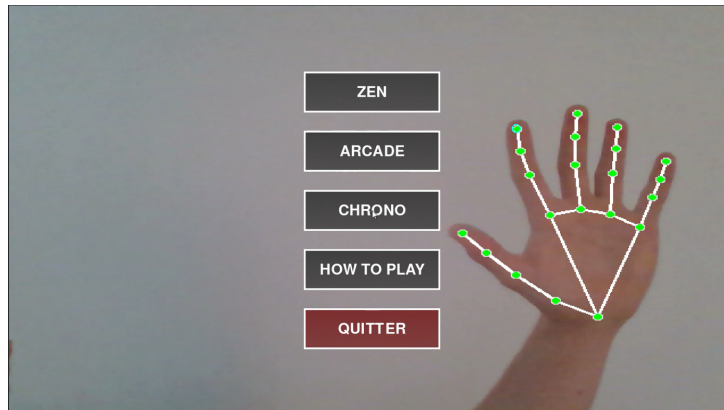


FIGURE 2 – Menu principal

## 6 Technologies et Algorithmes

### 6.1 Vision par Ordinateur

- **OpenCV** : capture webcam, conversion couleurs, redimensionnement.
- **MediaPipe Hands** : détection des landmarks de la main en temps réel. Fonctionne sur une image réduite à 320×240 pour minimiser la latence.
- **Flux optique Farneback** : calcul du mouvement dense entre deux frames. Utilisé pour créer une carte de magnitude.

### 6.2 Détection de Coupe

Deux méthodes combinées :

1. **Vitesse du curseur** : si l'index est proche (<150 px) d'un fruit et se déplace à plus de 250 px/s, la coupe est validée.
2. **Flux optique moyen** : moyenne de la magnitude du flux dans la bounding box du fruit. Un seuil de 0.4 (ajustable) déclenche la coupe.

### 6.3 Moteur de Jeu

- **Pygame** : rendu 2D, boucle événementielle, audio.
- Affichage vidéo en arrière-plan : conversion de l'image OpenCV en surface Pygame.
- Gestion des états : MENU, PLAYING, HOWTO.

## 6.4 Physique

- Gravité constante ( $600 \text{ px/s}^2$ ).
- Spawn multidirectionnel : depuis le bas (projection parabolique) ou depuis le haut (chute).
- Rotation, vitesse angulaire aléatoire.
- À la découpe, l'objet est remplacé par deux moitiés avec impulsion divergente.

## 7 Architecture Logicielle

---

SplashVision/

```
main.py          # Boucle principale, gestion des états, scaling
src/
  vision_engine.py # Classe VisionEngine : webcam, MediaPipe, flux optique
  game_manager.py  # Classe GameManager : spawn, scoring, particules, découpe
  objects.py       # Classes GameObject, Bomb, CutHalf, Particle
  ui.py            # Classes HoverButton, UI : menus, HUD, How-to
assets/
  sounds/          # slice.wav, boom.wav, hover.wav, click.wav, bip.wav, music.mp3
  images/          # (optionnel) sprites des fruits
  fonts/           # (optionnel) police .ttf
requirements.txt   # opencv-python, pygame, numpy, mediapipe
```

**Principe :** chaque module est indépendant et communique via des interfaces claires :

- VisionEngine fournit la position du curseur et la carte de magnitude.
- GameManager orchestre les objets et la logique de score.
- UI gère l'affichage et les interactions boutons, indépendamment du jeu.
- main.py assure la cohérence globale et le scaling.

## 8 Performances et Optimisations

---

- **Résolution adaptative** : le flux optique est calculé sur une image à 50% de la résolution de la webcam, puis redimensionné.
- **MediaPipe sur 320×240** : réduit drastiquement le temps d'inférence du modèle de main (environ 5–10 ms).
- **Pygame en mode FULLSCREEN** : évite les redimensionnements inutiles et tire parti de l'accélération matérielle.
- **Delta time** : tous les mouvements et timers utilisent  $\Delta t$  pour être insensibles au framerate.
- **Nettoyage régulier** : les objets hors écran et les particules mortes sont supprimés chaque frame.

**Résultat :** le jeu tourne à **60 FPS** constants sur une machine standard.

## 9 Retours Sensoriels

---

### 9.1 Visuels

- **Squelette de la main** : lignes blanches, points verts.
- **Particules** : petits cercles colorés éjectés à la découpe d'un fruit ou à l'explosion d'une bombe.
- **Screen Shake** : décalage aléatoire de quelques pixels pendant  $\sim 0,5$  s après une bombe.
- **Flash rouge** : surface semi-transparente rouge qui s'estompe rapidement.

## 9.2 Audio

- **Musique de fond** : music.mp3 jouée en boucle.
- **Effets** :
  - slice.wav – déchirement aigu à la coupe d'un fruit.
  - boom.wav – explosion grave pour les bombes.
  - hover.wav – léger son quand le curseur survole un bouton.
  - click.wav – validation d'un bouton.
  - bip.wav – toutes les secondes dans les 10 dernières secondes du Chrono.

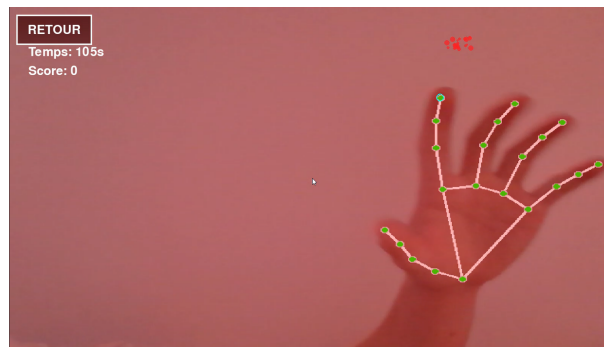


FIGURE 3 – Effet visuel d'une bombe

## 10 Guide de Prise en Main

1. **Lancement** : python main.py (plein écran).
2. **Menu** : pointez un bouton avec l'index et restez immobile 2 secondes.
3. **En jeu** : des fruits apparaissent. Passez la main dessus pour les couper.
4. **Retour** : visez le bouton « RETOUR » en haut à gauche (maintien 1,5 s).
5. **Quitter** : utilisez le bouton « QUITTER » du menu.

### Conseils pour une bonne détection :

- Assurez-vous d'avoir un éclairage suffisant et un fond contrasté.
- Gardez la main à une distance raisonnable de la caméra.
- Si la main n'est pas détectée, le curseur suit le mouvement global (agitez la main).

## 11 Personnalisation et Extensions

Le code est conçu pour être facilement modifiable :

**Seuils de découpe** (dans game\_manager.py) :

```
self.slice_max_distance = 150
self.slice_cursor_speed = 250
self.slice_motion_backup = 0.4
```

**Temps de maintien des boutons** (dans main.py) :

```
HOLD_TIME_MENU = 1.8
HOLD_TIME_BACK = 1.5
```

### Extensions possibles :

- **Ajout de modes** : créer une nouvelle lambda dans setup\_menu() et étendre game\_manager.py.
- **Nouveaux gestes** : exploiter d'autres landmarks de la main (ex. poing fermé = pause).

## 12 Démonstration Visuelle

---

Les captures d'écran suivantes illustrent les principaux écrans du jeu (à insérer dans la version finale du rapport) :

1. **Menu principal** avec les 5 boutons.
2. **Écran de jeu** avec le squelette de la main, un fruit en train d'être coupé, et les particules.
3. **Effet de bombe** : flash rouge et écran tremblé.
4. **Écran How-to** avec les instructions.

## 13 Conclusion et Perspectives

---

SplashVision prouve qu'il est possible de créer un jeu vidéo interactif en utilisant uniquement une webcam et des bibliothèques open source. Il mélange avec succès **vision par ordinateur**, **game design** et **expérience utilisateur** pour offrir une nouvelle forme de divertissement sans contact.

### Futures évolutions possibles :

- Support multi-mains (deux joueurs simultanés).
- Reconnaissance de gestes plus complexes (balayage, cercle).
- Mode multijoueur en réseau.
- Calibration automatique du seuil de sensibilité.
- Interface graphique plus riche avec sprites animés.
- Portage mobile grâce à une webcam externe.

## 14 Remerciements et Crédits

---

- **MediaPipe** par Google – détection de la main.
- **OpenCV** – traitement d'images.
- **Pygame** – moteur de jeu.
- **NumPy** – calcul matriciel.

**SplashVision** – *Votre main est la seule manette dont vous avez besoin.*